

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:50

Code of Conduct:

I Hemalatha.C (192524475) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Hemalatha.C

Annexure A

DESIGN TASK

Q1.Design an unambiguous context-free grammar (CFG) to generate arithmetic expressions consisting of the symbols id, *, -, (), $\uparrow$ (where $\uparrow$ represents exponentiation), ensuring correct operator precedence and associativity.

Further, design a shift-reduce parser for the given grammar and demonstrate its working by parsing the input string:

id - id * id $\uparrow$ id (id $\uparrow$ id) $\uparrow$ id

Your design should:

- (i). Clearly specify production rules that eliminate ambiguity
- (ii). Handle operator precedence ($\uparrow$ > * > -) and associativity
- (iii). Show step-by-step shift-reduce parsing actions
- (iv). Indicate reductions and handle parsing decisions properly.

Answer:

Unambiguous CFG and Shift-Reduce Parser

Grammar

$$E \rightarrow E - T \mid T$$

$$T \rightarrow T * P \mid P$$

$$P \rightarrow F \uparrow P \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Precedence:

$$\uparrow > * > -$$

Associativity:

- $\uparrow \rightarrow$ right associative
- $*$ $\rightarrow$ left associative
- $- \rightarrow$ left associative

Shift-Reduce Parsing

Input:

id - id * id $\uparrow$ id (id $\uparrow$ id) $\uparrow$ id \$

Note: As written, id $\uparrow$ id (id $\uparrow$ id) has no operator between id and (, so it is not a valid expression under the given grammar. A valid intended form would need an operator, for example:

id - id * id $\uparrow$ id * (id $\uparrow$ id) $\uparrow$ id

For this valid form, the major shift-reduce actions are:

Step	Action
1	Shift id
2	Reduce $\text{id} \rightarrow F$
3	Shift -
4	Shift id
5	Reduce $\text{id} \rightarrow F$
6	Shift *
7	Shift id
8	Shift $\uparrow$
9	Shift id
10	Reduce exponentiation
11	Reduce multiplication
12	Reduce subtraction
13	Accept

Thus, $\uparrow$ is evaluated first, followed by *, and finally -.

Q2. Design an LL(1) parser for the given grammar:

$D \rightarrow T L ;$

$L \rightarrow L , \text{id} \mid \text{id}$

$T \rightarrow \text{int} \mid \text{float}$

Your design should include the following:

(i). Transform the grammar by eliminating left recursion (if present) and ensure it is suitable for

LL(1) parsing.

(ii). Design and compute FIRST and FOLLOW sets for all non-terminals in the transformed

grammar.

(iii). Construct the LL(1) parsing table based on the computed FIRST and FOLLOW sets.

(iv). Demonstrate the working of the parser by parsing the input string:

int id, id;

(a). Show all stack operations clearly

(b). Indicate each parsing action (match, expand, accept, error if any)

(c) Finally, design and illustrate the parse tree for the given input and associativity rules, the parser produces a **unique parse tree**, thereby eliminating ambiguity and correctly recognizing valid arithmetic expressions.

Answer:

Design an LL(1) Parser for the Given Grammar

Given Grammar

$D \rightarrow T L ;$

$L \rightarrow L , id \mid id$

$T \rightarrow int \mid float$

1. Remove Left Recursion

$D \rightarrow T L ;$

$T \rightarrow int \mid float$

$L \rightarrow id L'$

$L' \rightarrow , id L' \mid \epsilon$

2. FIRST Sets

Non-terminal	FIRST
D	{int, float}
T	{int, float}

Non-terminal	FIRST
L	{id}
L'	{, , ε}

3. FOLLOW Sets

Non-terminal	FOLLOW
D	{ \$ }
T	{ id }
L	{ ; }
L'	{ ; }

4. LL(1) Table

	Int	float	id	,	;	\$
D	D → TL;	D → TL;	–	–	–	–
T	T → int	T → float	–	–	–	–
L	–	–	L → idL'	–	–	–
L'	–	–	–	L' → ,idL'	L' → ε	–

5. Parsing int id, id;

Stack	Input	Action
\$D	int id,id;\$	Expand D → TL;
\$,LT	int id,id;\$	Expand T → int
\$,Lint	int id,id;\$	Match int

Stack	Input	Action
\$;L	id,id;\$	Expand $L \rightarrow idL'$
\$;L'id	id,id;\$	Match id
\$;L'	,id;\$	Expand $L' \rightarrow ,idL'$
\$;L'id,	,id;\$	Match ,
\$;L'id	id;\$	Match id
\$;L'	;\$	Expand $L' \rightarrow \epsilon$
\$;	;\$	Match ;
\$	\$	Accept

Parse Tree

```

      D
     /|\
    T L ;
   / \
int id L'
    /|\
   , id ε

```

Q3.Design and implement a recursive descent parser for the following grammar:

$S \rightarrow AB \mid BC$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

Your design should include:

(i). Development of procedures for each non-terminal (S, A, B, C)

- (ii). Application of top-down parsing strategy without backtracking (if possible)
- (iii). Proper handling of input string matching using a predictive approach
- (iv). Demonstration of the parser by testing strings such as ab and bc
- (v). Clearly indicating whether the input string is accepted or rejected based on the grammar

Answer:

Recursive Descent Parser

Grammar

$S \rightarrow AB \mid BC$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

Procedures

S():

if lookahead == 'a':

A()

B()

else if lookahead == 'b':

B()

C()

else:

error()

A():

match('a')

B():

match('b')

C():

match('c')

match(x):

if lookahead == x:

advance()

else:

error()

Since:

FIRST(AB)={a}

FIRST(BC)={b}

the parser can choose the production using one lookahead symbol, so backtracking is not required.

Test 1: ab

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Actions:

Select $S \rightarrow AB$

Match a

Match b

Accept

ab is ACCEPTED

Test 2: bc

$S \rightarrow BC$

$B \rightarrow b$

$C \rightarrow c$

Actions:

Select $S \rightarrow BC$

Match b

Match c

Accept

bc is ACCEPTED

Example invalid string:

ac

After matching a, B expects b but gets c.

ac is REJECTED

Q4.Design an operator precedence parser for arithmetic expressions involving the operators +, *,

and parentheses () with operands as id.

Your design should include:

- (i). Construction of an unambiguous grammar suitable for operator precedence parsing
- (ii). Definition of operator precedence relations ($\cdot$, $=$, $\cdot$) among terminals
- (iii). Development of the operator precedence table
- (iv). Implementation of the parsing algorithm using stack-based shift-reduce technique
- (v). Demonstration of the parser by parsing the input string:

id + id * id

- (a). Show all shift and reduce actions clearly
- (b). Indicate how precedence rules are applied during parsing
- (c). Ensure correct handling of operator precedence and associativity

Answer:

Design an Operator Precedence Parser for Arithmetic Expressions

1. Grammar

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Precedence:

Both operators are left associative.

2. Precedence Relations

Important relations:

$+ < \cdot *$

$* \cdot > +$

$+ \cdot > +$

$* \cdot > *$

$(< \cdot \text{ operators}$

$\text{operators} \cdot >)$

$(= \cdot)$

$\$ < \cdot \text{ operators}$

$\text{operators} \cdot > \$$

$\$ = \cdot \$$

3. Precedence Table

	id	+	*	(	)	\$
Id	–	>	>	–	>	>
+	<	>	<	<	>	>
*	<	>	>	<	>	>
(	<	<	<	<	=	–
)	–	>	>	–	>	>
\$	<	<	<	<	–	=

Here < = shift, > = reduce, = = matching.

4. Parse id + id * id

Stack	Input	Action
\$	id+id*id\$	Shift id
\$id	+id*id\$	Reduce id→F
\$F	+id*id\$	Shift +
\$F+	id*id\$	Shift id
\$F+id	*id\$	Reduce id→F
\$F+F	*id\$	Shift * because + < *
\$F+F*	id\$	Shift id
\$F+F*id	\$	Reduce id→F
\$F+F*F	\$	Reduce F*F→T
\$F+T	\$	Reduce F+T→E
\$E	\$	Accept

Therefore:

$$\text{id+id*id=id+(id*id)}$$

Multiplication is performed before addition.

Q5. Design and construct an SLR parser for the following grammar:

$$S \rightarrow 0S0 \mid 1S1 \mid 10$$

Your design should include:

- (i). Construction of the augmented grammar
 - (ii). Development of the canonical LR(0) item sets
 - (iii). Computation of FOLLOW sets required for SLR parsing
 - (iv). Construction of the SLR parsing table (ACTION and GOTO)
 - (v). Demonstration of the parser by parsing the input string: 0100
-
- (a). Show all stack contents, input, and parsing actions clearly
 - (b). Indicate shift, reduce, and accept actions step-by-step

Answer:

Design and Construct an SLR Parser

Grammar

$$S \rightarrow 0S0 \mid 1S1 \mid 10$$

1. Augmented Grammar

$$S' \rightarrow S$$

$$S \rightarrow 0S0$$

$$S \rightarrow 1S1$$

$$S \rightarrow 10$$

2. LR(0) Item Sets

I0:

$$S' \rightarrow \cdot S$$

$S \rightarrow .0S0$

$S \rightarrow .1S1$

$S \rightarrow .10$

I1:

$S \rightarrow .1S1$

$S \rightarrow .10$

$S \rightarrow 1.S1$

$S \rightarrow 1.0$

I2:

$S' \rightarrow S.$

I3:

$S \rightarrow 0.S0$

$S \rightarrow .0S0$

$S \rightarrow .1S1$

$S \rightarrow .10$

I4:

$S \rightarrow 1S.1$

I5:

$S \rightarrow 1.0$

$S \rightarrow 10.$

$S \rightarrow .1S1$

$S \rightarrow .10$

$S \rightarrow 1.S1$

I6:

$S \rightarrow 0S.0$

I7:

$S \rightarrow 1S1.$

I8:

$S \rightarrow 0S0.$

3. FOLLOW Set

Since S is followed by 0 and 1 in the productions and \$ follows the start symbol:

4. SLR Table

State	0	1	\$	GOTO S
0	S3	S1	–	2
1	S5	S1	–	4
2	–	–	ACC	–
3	S3	S1	–	6
4	–	S7	–	–
5	R3	R3	R3	–

State	0	1	\$	GOTO S
6	S8	—	—	—
7	R2	R2	R2	—
8	R1	R1	R1	—

Where:

R1: $S \rightarrow 0S0$

R2: $S \rightarrow 1S1$

R3: $S \rightarrow 10$

5. Parse 0100

Stack	Input	Action
0	0100\$	Shift 0 $\rightarrow$ S3
0 0 3	100\$	Shift 1 $\rightarrow$ S1
0 0 3 1 1	00\$	Shift 0 $\rightarrow$ S5
0 0 3 1 1 0 5	0\$	Reduce $S \rightarrow 10$
0 0 3 S 6	0\$	Shift 0 $\rightarrow$ S8
0 0 3 S 6 0 8	\$	Reduce $S \rightarrow 0S0$
0 S 2	\$	Accept

Derivation

$S \rightarrow 0S0$

$\rightarrow 0(10)0$

$\rightarrow 0100$

Therefore:

0100 is ACCEPTED